ASoC

ASoC ASoC, ALSA System on Chip: is a Linux kernel subsystem

created to provide better ALSA support for system-on-chip and portable audio codecs. It allows to reuse codec drivers across multiple architectures and provides an API to integrate them with the SoC audio interface.

- created for that use case
- designed for codec drivers reuse
- has an API to write codec drivers
- has an API to write SoC interface drivers

ASoC components

- Codec class drivers: define the codec capabilities (audio interface, audio controls, analog inputs and outputs).
- Platform class drivers: defines the SoC audio interface (also referred as CPU DAI), sets up DMA when applicable.
- Codec to platform integration: nowadays, usually done through device tree, previously required writing a machine driver in C.

Note: The codec can be part of another IC (PMIC, Bluetooth or MODEM chips).

simple-audio-card

simple-card Most sound cards, can now be described

using device tree. This is done using a sound node with a

```
simple-audio-card
```

compatible string.

- The DT bindings are documented in
- The driver handling it is

Since 2017, OF-graph based bindings are available.

- They are documented in
- The driver handling it is

Both required a few changes in the SoC DAI drivers to be usable for example to select the audio mode for the SSC on Microchip SoCs or configure properly the i.MX audmux.

simple-card - example 1 Let's say we have an ADAU1372

codec connected to an i.Mx6UL SAI. First, enable the SAI and the codec:

```
&sai2 {
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_sai2>;
        status = "okay";
};

&i2c1 {
        adau1372: codec@3c {
                #sound-dai-cells = <0>;
                compatible = "adi,adau1372";
                reg = <0x3c>;
                clock-names = "mclk";
                clocks = <&adau1372z_xtal>;
        };
};
```

simple-card - example 1 Let's say we have an ADAU1372

```
/ {
        adau1372z_xtal: adau1372z_xtal {
                compatible = "fixed-clock";
                #clock-cells = <0>;
                clock-frequency = <12288000>;
        };
};
```

simple-card - example 1 Now, describe the sound card:

```
sound {
        compatible = "simple-audio-card";
        simple-audio-card,name = "imx6ul-adau1372";

        simple-audio-card,dai-link@0 {
                format = "i2s";
                bitclock-master = <&adau1372_dai>;
                frame-master = <&adau1372_dai>;

                sai2_dai: cpu {
                        sound-dai = <&sai2>;
                };

                adau1372_dai: codec {
                        sound-dai = <&adau1372>;
                };
```

simple-card - example 1 Now, describe the sound card:

```
                    };
        };
```

For convenience, the codec is the producer, it generates both BCLK and FSCLK.

simple-card - example 2 The ADAU1372 has actually 4 channels and can do TDM:

```
sound {
        compatible = "simple-audio-card";
        simple-audio-card,name = "imx6ul-adau1372";

        simple-audio-card,dai-link@0 {
                format = "i2s";
                bitclock-master = <&adau1372_dai>;
                frame-master = <&adau1372_dai>;

                sai2_dai: cpu {
                        sound-dai = <&sai2>;
                        dai-tdm-slot-num = <4>;
                        dai-tdm-slot-width = <32>;
                };
```

simple-card - example 2 The ADAU1372 has actually 4

```
                        adau1372_dai: codec {
                                sound-dai = <&adau1372>;
                                dai-tdm-slot-num = <4>;
                                dai-tdm-slot-width = <32>;
                        };
                };
        };
```

simple-card - example 3 However, the ADAU1372 has an

hardware issue and doesn't generate the proper BCLK when doing TDM4 with a 32kHz sample rate. The SAI has to be master:

```
sound {
        compatible = "simple-audio-card";
        simple-audio-card,name = "imx6ul-adau1372";

        simple-audio-card,dai-link@0 {
                format = "i2s";
                bitclock-master = <&sai2_dai>;
                frame-master = <&sai2_dai>;

                sai2_dai: cpu {
                        sound-dai = <&sai2>;
                        dai-tdm-slot-num = <4>;
                        dai-tdm-slot-width = <32>;
```

simple-card - example 3 However, the ADAU1372 has an

```
                        };

                        adau1372_dai: codec {
                                sound-dai = <&adau1372>;
                                dai-tdm-slot-num = <4>;
                                dai-tdm-slot-width = <32>;
                        };
                };
        };
```

simple-card - example 3 The result is not what is

expected:

```
    # aplay test.wav
    Playing WAVE 'test.wav' :
    Signed 16 bit Little Endian, Rate  32000 Hz, Stereo
    aplay: set_params:1403: Unable to install hw params:
    [...]
    # dmesg
    [...]
    fsl-sai 202c000.sai: failed to derive required Tx rate: 4096000
    fsl-sai 202c000.sai: ASoC: can't set 202c000.sai hw params: -22
    # cat /sys/kernel/debug/clk/clk_summary
    pll3                              1        1        0   480000000
0     0  50000
       pll3_bypass                    1        1        0   480000000
0     0  50000
           pll3_usb_otg               2        3        0   480000000
```

simple-card - example 3 The result is not what is

```
0     0  50000
              pll3_pfd2_508m          0        0        0   508235294
0     0  50000
                 sai2_sel             0        0        0   508235294
0     0  50000
                    sai2_pred         0        0        0   127058824
0     0  50000
                       sai2_podf      0        0        0    63529412
0     0  50000
                          sai2        0        0        0    63529412
0     0  50000
```

Indeed, there is no way for the SAI to divide 63529412 to get the proper BCLK!

device tree - clocks It is possible to reparent clocks

using `assigned-clock-parents` and set the clock rate using

```
assigned-clock-rates
```

.

```
&sai2 {
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_sai2>;
        assigned-clocks = <&clks IMX6UL_CLK_SAI2_SEL>, <&clks
IMX6UL_CLK_SAI2>;
        assigned-clock-parents = <&clks IMX6UL_CLK_PLL4_AUDIO_DIV>;
        assigned-clock-rates = <196608000>, <24576000>;
        status = "okay";
};
```

device tree - clocks It is possible to reparent clocks

Notice that 24.576MHz was selected for the sai input clock as it is not able to divide by 3 to obtain the 4.096MHz BCLK.

simple-card - example 4 There is a possible cost

reduction, the SAI is able to output its clock to feed to the codec MCLK instead of the crystal:

```
&sai2 {
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_sai2>;
        fsl,sai-mclk-direction-output;
        status = "okay";
};

&i2c1 {
        adau1372: codec@3c {
                #sound-dai-cells = <0>;
                compatible = "adi,adau1372";
                reg = <0x3c>;
                clock-names = "mclk";
```

simple-card - example 4 There is a possible cost

```
                clocks = <&clks IMX6UL_CLK_SAI2>;
                assigned-clocks = <&clks IMX6UL_CLK_SAI2_SEL>, <&clks
IMX6UL_CLK_SAI2>;
                assigned-clock-parents = <&clks IMX6UL_CLK_PLL4_AUDIO_DIV>;
                assigned-clock-rates = <196608000>, <24576000>;
        };
};
```

This replaces the 12.288MHz crystal by the 24.576 MCLK from the SAI. This works because the codec has a configurable divider for MCLK and can divide by 2. Also the clock parents and rates assignment has moved to the codec because of probing order.

simple-card - routing It is possible but not mandatory

to list the actual audio connections present on the board, this is called routing. The first step is to define the board connectors, in this case two stereo line input jack (Line0 and Line1) and a stereo jack output.

```
simple-audio-card,widgets =
        "Line", "Line0",
        "Line", "Line1",
        "Headphone", "Headphone Jack",
```

simple-card - routing Routing audio from the codec to

the board connector is then done using `simple-audio-card,routing`

```
simple-audio-card,routing =
        "AIN0", "Line0",
        "AIN1", "Line0",
        "AIN2", "Line1",
        "AIN3", "Line1",
        "Headphone Jack", "HPOUTL",
        "Headphone Jack", "HPOUTR",
```

Look for `SND_SOC_DAPM_OUTPUT` and `SND_SOC_DAPM_INPUT` to know what the codec is providing.

ASoC

simple-card - routing Routing audio from the codec to

```
int snd_soc_register_card(struct snd_soc_card *card);
int snd_soc_unregister_card(struct snd_soc_card *card);
int devm_snd_soc_register_card(struct device *dev, struct snd_soc_card
*card);
[...]
/* SoC card */
struct snd_soc_card {
        const char *name;
        const char *long_name;
        const char *driver_name;
        struct device *dev;
        struct snd_card *snd_card;
[...]
        /* CPU <--> Codec DAI links  */
        struct snd_soc_dai_link *dai_link;  /* predefined links only */
        int num_links;  /* predefined links only */
```

simple-card - routing Routing audio from the codec to

```
        struct list_head dai_link_list; /* all links */
        int num_dai_links;
[...]
};
```

simple-card - routing Routing audio from the codec to the codec DAI.

```
struct snd_soc_dai_link {
        /* config - must be set by machine driver */
        const char *name;                       /* Codec name */
        const char *stream_name;               /* Stream name */

        /*
         * You MAY specify the link's CPU-side device, either by device
name,
         * or by DT/OF node, but not both. If this information is omitted,
         * the CPU-side DAI is matched using .cpu_dai_name only, which
hence
         * must be globally unique. These fields are currently typically
used
         * only for codec to codec links, or systems using device tree.
         */
```

simple-card - routing Routing audio from the codec to

```
        /*
         * You MAY specify the DAI name of the CPU DAI. If this information
is
         * omitted, the CPU-side DAI is matched
using .cpu_name/.cpu_of_node
         * only, which only works well when that device exposes a single
DAI.
         */
        struct snd_soc_dai_link_component *cpus;
        unsigned int num_cpus;
```

simple-card - routing Routing audio from the codec to

```
        /*
         * You MUST specify the link's codec, either by device name, or by
         * DT/OF node, but not both.
         */
        /* You MUST specify the DAI name within the codec */
        struct snd_soc_dai_link_component *codecs;
        unsigned int num_codecs;
[...]
        unsigned int dai_fmt;           /* format to set on init */
[...]
}
```

simple-card - routing Routing audio from the codec to

```
SND_SOC_DAILINK_DEFS(pcm,
        DAILINK_COMP_ARRAY(COMP_EMPTY()),
        DAILINK_COMP_ARRAY(COMP_CODEC(NULL, "wm8904-hifi")),
        DAILINK_COMP_ARRAY(COMP_EMPTY()));

static struct snd_soc_dai_link atmel_asoc_wm8904_dailink = {
        .name = "WM8904",
        .stream_name = "WM8904 PCM",
        .dai_fmt = SND_SOC_DAIFMT_I2S
                | SND_SOC_DAIFMT_NB_NF
                | SND_SOC_DAIFMT_CBP_CFP,
        .ops = &atmel_asoc_wm8904_ops,
        SND_SOC_DAILINK_REG(pcm),
};

static struct snd_soc_card atmel_asoc_wm8904_card = {
```

simple-card - routing Routing audio from the codec to

```
        .name = "atmel_asoc_wm8904",
        .owner = THIS_MODULE,
        .dai_link = &atmel_asoc_wm8904_dailink,
        .num_links = 1,
        .dapm_widgets = atmel_asoc_wm8904_dapm_widgets,
        .num_dapm_widgets = ARRAY_SIZE(atmel_asoc_wm8904_dapm_widgets),
        .fully_routed = true,
};
```

simple-card - routing Routing audio from the codec to

```
static int atmel_asoc_wm8904_dt_init(struct platform_device *pdev)
{
        struct device_node *np = pdev->dev.of_node;
        struct device_node *codec_np, *cpu_np;
        struct snd_soc_card *card = &atmel_asoc_wm8904_card;
        struct snd_soc_dai_link *dailink = &atmel_asoc_wm8904_dailink;
[...]
        cpu_np = of_parse_phandle(np, "atmel,ssc-controller", 0);
        if (!cpu_np) {
                dev_err(&pdev->dev, "failed to get dai and pcm infon");
                ret = -EINVAL;
                return ret;
        }
        dailink->cpus->of_node = cpu_np;
        dailink->platforms->of_node = cpu_np;
        of_node_put(cpu_np);
```

simple-card - routing Routing audio from the codec to

```
codec_np = of_parse_phandle(np, "atmel,audio-codec", 0);
if (!codec_np) {
        dev_err(&pdev->dev, "failed to get codec infon");
        ret = -EINVAL;
        return ret;
}
dailink->codecs->of_node = codec_np;
of_node_put(codec_np);
```

simple-card - routing Routing audio from the codec to

```
static int atmel_asoc_wm8904_probe(struct platform_device *pdev)
{
        struct snd_soc_card *card = &atmel_asoc_wm8904_card;
        struct snd_soc_dai_link *dailink = &atmel_asoc_wm8904_dailink;
        int id, ret;

        card->dev = &pdev->dev;
        ret = atmel_asoc_wm8904_dt_init(pdev);
        if (ret) {
                dev_err(&pdev->dev, "failed to init dt infon");
                return ret;
        }

        id = of_alias_get_id((struct device_node *)dailink->cpus->of_node,
"ssc");
        ret = atmel_ssc_set_audio(id);
```

simple-card - routing Routing audio from the codec to

```
        if (ret != 0) {
                dev_err(&pdev->dev, "failed to set SSC %d for audion", id);
                return ret;
        }

        ret = snd_soc_register_card(card);
        if (ret) {
                dev_err(&pdev->dev, "snd_soc_register_card failedn");
                goto err_set_audio;
        }
[...]
}
```

simple-card - routing Routing audio from the codec to

DAI driver, it is still necessary to define what are the codec outputs and inputs that are actually used on the board. This is called routing.

- statically: using the `.dapm_routes` and `.num_dapm_routes` members of
- from device tree:

```
int snd_soc_of_parse_audio_routing(struct snd_soc_card *card,
                                   const char *propname);
```

simple-card - routing Routing audio from the codec to

```
static const struct snd_soc_dapm_route rk_audio_map[] = {
        {"IN34", NULL, "Headset Mic"},
        {"IN34", NULL, "MICBIAS"},
        {"Headset Mic", NULL, "MICBIAS"},
        {"DMICL", NULL, "Int Mic"},
        {"Headphone", NULL, "HPL"},
        {"Headphone", NULL, "HPR"},
        {"Speaker", NULL, "SPKL"},
        {"Speaker", NULL, "SPKR"},
};
[...]
static struct snd_soc_card snd_soc_card_rk = {
        .name = "ROCKCHIP-I2S",
        .owner = THIS_MODULE,
        .dai_link = &rk_dailink,
        .num_links = 1,
```

simple-card - routing Routing audio from the codec to

```
[...]
        .dapm_widgets = rk_dapm_widgets,
        .num_dapm_widgets = ARRAY_SIZE(rk_dapm_widgets),
        .dapm_routes = rk_audio_map,
        .num_dapm_routes = ARRAY_SIZE(rk_audio_map),
        .controls = rk_mc_controls,
        .num_controls = ARRAY_SIZE(rk_mc_controls),
};
```

simple-card - routing Routing audio from the codec to

```
static int atmel_asoc_wm8904_dt_init(struct platform_device *pdev)
{
[...]
        ret = snd_soc_of_parse_card_name(card, "atmel,model");
        if (ret) {
                dev_err(&pdev->dev, "failed to parse card namen");
                return ret;
        }

        ret = snd_soc_of_parse_audio_routing(card, "atmel,audio-routing");
        if (ret) {
                dev_err(&pdev->dev, "failed to parse audio routingn");
                return ret;
        }
```

simple-card - routing Routing audio from the codec to

```
[...]
}
```

simple-card - routing Routing audio from the codec to

```
  - atmel,audio-routing: A list of the connections between audio
components.
    Each entry is a pair of strings, the first being the connection's sink,
    the second being the connection's source. Valid names for sources and
    sinks are the WM8904's pins, and the jacks on the board:

    WM8904 pins:

    * IN1L
    * IN1R
    * IN2L
    * IN2R
    * IN3L
    * IN3R
    * HPOUTL
    * HPOUTR
```

simple-card - routing Routing audio from the codec to

```
* LINEOUTL
* LINEOUTR
* MICBIAS

Board connectors:

* Headphone Jack
* Line In Jack
* Mic
```

simple-card - routing Routing audio from the codec to

```
Example:
sound {
        compatible = "atmel,asoc-wm8904";
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_pck0_as_mck>;

        atmel,model = "wm8904 @ AT91SAM9N12EK";

        atmel,audio-routing =
                "Headphone Jack", "HPOUTL",
                "Headphone Jack", "HPOUTR",
                "IN2L", "Line In Jack",
                "IN2R", "Line In Jack",
                "Mic", "MICBIAS",
                "IN1L", "Mic";
```

simple-card - routing Routing audio from the codec to

```
        atmel,ssc-controller = <&ssc0>;
        atmel,audio-codec = <&wm8904>;
};
```

simple-card - routing Routing audio from the codec to

defined in the codec driver. Look for the `SND_SOC_DAPM_INPUT` and `SND_SOC_DAPM_OUTPUT` definitions.

```
static const struct snd_soc_dapm_widget wm8904_adc_dapm_widgets[] = {
SND_SOC_DAPM_INPUT("IN1L"),
SND_SOC_DAPM_INPUT("IN1R"),
SND_SOC_DAPM_INPUT("IN2L"),
SND_SOC_DAPM_INPUT("IN2R"),
SND_SOC_DAPM_INPUT("IN3L"),
SND_SOC_DAPM_INPUT("IN3R"),
[...]
};

static const struct snd_soc_dapm_widget wm8904_dac_dapm_widgets[] = {
[...]
SND_SOC_DAPM_OUTPUT("HPOUTL"),
```

simple-card - routing Routing audio from the codec to

```
SND_SOC_DAPM_OUTPUT("HPOUTR"),
SND_SOC_DAPM_OUTPUT("LINEOUTL"),
SND_SOC_DAPM_OUTPUT("LINEOUTR"),
};
```

simple-card - routing Routing audio from the codec to

defined in the machine driver, in the part of the registered .

```
static const struct snd_soc_dapm_widget atmel_asoc_wm8904_dapm_widgets[] =
{
        SND_SOC_DAPM_HP("Headphone Jack", NULL),
        SND_SOC_DAPM_MIC("Mic", NULL),
        SND_SOC_DAPM_LINE("Line In Jack", NULL),
};
```

simple-card - routing Routing audio from the codec to

relationship is declared part of the `.dai_fmt` field of .

```
/*
 * DAI hardware clock providers/consumers
 *
 * This is wrt the codec, the inverse is true for the interface
 * i.e. if the codec is clk and FRM provider then the interface is
 * clk and frame consumer.
 */
#define SND_SOC_DAIFMT_CBP_CFP                (1 << 12) /* codec clk
provider & frame provider */
#define SND_SOC_DAIFMT_CBC_CFP                (2 << 12) /* codec clk
consumer & frame provider */
#define SND_SOC_DAIFMT_CBP_CFC                (3 << 12) /* codec clk
provider & frame consumer */
#define SND_SOC_DAIFMT_CBC_CFC                (4 << 12) /* codec clk
consumer & frame consumer */
```

simple-card - routing Routing audio from the codec to

```
/* previous definitions kept for backwards-compatibility, do not use in new
contributions */
#define SND_SOC_DAIFMT_CBM_CFM                SND_SOC_DAIFMT_CBP_CFP
#define SND_SOC_DAIFMT_CBS_CFM                SND_SOC_DAIFMT_CBC_CFP
#define SND_SOC_DAIFMT_CBM_CFS                SND_SOC_DAIFMT_CBP_CFC
#define SND_SOC_DAIFMT_CBS_CFS                SND_SOC_DAIFMT_CBC_CFC
```

```
        .dai_fmt = SND_SOC_DAIFMT_I2S
                 | SND_SOC_DAIFMT_NB_NF
                 | SND_SOC_DAIFMT_CBM_CFM,
```

simple-card - routing Routing audio from the codec to

member of contains useful callbacks.

```
/* SoC audio ops */
struct snd_soc_ops {
        int (*startup)(struct snd_pcm_substream *);
        void (*shutdown)(struct snd_pcm_substream *);
        int (*hw_params)(struct snd_pcm_substream *, struct
snd_pcm_hw_params *);
        int (*hw_free)(struct snd_pcm_substream *);
        int (*prepare)(struct snd_pcm_substream *);
        int (*trigger)(struct snd_pcm_substream *, int);
};
```

`.hw_params` is called when setting up the audio stream. The contains the audio characteristics. Use `params_rate()` to get the sample rate, `params_channels` for the number of channels and

simple-card - routing Routing audio from the codec to

```
params_format
```

to get the format (including the bit depth). Finally, `snd_soc_params_to_bclk` calculates the bit clock.

simple-card - routing Routing audio from the codec to

- `params_rate` gets the sample rate
- `params_channels` gets the number of channels
- `params_format` gets the format (including the bit depth)
- `snd_soc_params_to_bclk` calculates the bit clock.
- `snd_soc_dai_set_sysclk` sets the clock rate and direction for the DAI (SoC or codec)

```
int snd_soc_dai_set_sysclk(struct snd_soc_dai *dai, int clk_id,
        unsigned int freq, int dir);
```

- it is also possible to configure the PLLs and clock divisors if necessary

```
int snd_soc_dai_set_clkdiv(struct snd_soc_dai *dai,
        int div_id, int div);
int snd_soc_dai_set_pll(struct snd_soc_dai *dai,
        int pll_id, int source, unsigned int freq_in, unsigned int freq_out);
```

simple-card - routing Routing audio from the codec to

```
static int atmel_asoc_wm8904_hw_params(struct snd_pcm_substream *substream,
                struct snd_pcm_hw_params *params)
{
        struct snd_soc_pcm_runtime *rtd = substream->private_data;
        struct snd_soc_dai *codec_dai = rtd->codec_dai;
        int ret;

        ret = snd_soc_dai_set_pll(codec_dai, WM8904_FLL_MCLK,
WM8904_FLL_MCLK,
                32768, params_rate(params) * 256);
        if (ret < 0) {
                pr_err("%s - failed to set wm8904 codec PLL.", __func__);
                return ret;
        }
```

simple-card - routing Routing audio from the codec to

```
        /*
         * As here wm8904 use FLL output as its system clock
         * so calling set_sysclk won't care freq parameter
         * then we pass 0
         */
        ret = snd_soc_dai_set_sysclk(codec_dai, WM8904_CLK_FLL,
                                     0, SND_SOC_CLOCK_IN);
        if (ret < 0) {
                pr_err("%s -failed to set wm8904 SYSCLKn", __func__);
                return ret;
        }

        return 0;
}

static struct snd_soc_ops atmel_asoc_wm8904_ops = {
```

simple-card - routing Routing audio from the codec to

```
        .hw_params = atmel_asoc_wm8904_hw_params,
};
```

simple-card - routing Routing audio from the codec to